

QR Secures Security Analysis for QR Codes and Web Addresses

Capstone

WINTER 2026

Supervisor: Dr. ADNAN

Team Members

Manea Al Shabrain (Leader)
Sulaiman Abdo
Saeed Alsawaf
Mohammad Albustami
Salah Alharbi

Application

<https://qrsecures.base44.app>

Date

13/Sep/2026

Abstract

QR codes make it easy to connect printed material with online services, but they also hide the destination from the user. Attackers can take advantage of that gap by placing phishing, scam, or malware links inside a QR code. QR Secures was developed to examine the destination before the user opens it. The work began during the practicum with a character level Convolutional Neural Network for malicious URL classification, supported by URL rules and SSL certificate checks. The capstone expanded the project into a Base44 application that supports camera scanning, manual URL checks, explainable risk results, scan history, dashboards, community reports, threat hunting, investigations, watchlists, user roles, and Slack alerts. The practicum report recorded an accuracy of 0.982 and a ROC AUC of 0.995 for the CNN. It also recorded 48 correct decisions from 50 difficult URLs for the weighted hybrid detector. These numbers belong to the practicum system and are not presented as measurements of the later language model application. The capstone contribution is the design and implementation of a complete security workflow that brings scanning, analysis, reporting, and response into one interface. The application gives people more information before they visit an unknown address and gives analysts a practical place to review related activity. Source export from Base44 was unavailable under the current plan, so some application details remain documented behavior rather than independently reviewed code. The report states those limits clearly and identifies the security work still required before production use.

Keywords

QR code security, malicious URL detection, phishing, language model analysis, SSL inspection, redirect analysis, community reporting, threat hunting, incident response

Table of Contents

1	Introduction
2	Project Scope
3	Background and Related Work
4	Project Development
5	System Requirements
6	System Architecture
7	Application Features
8	Data Design
9	Scan and Analysis Workflow
10	Community Intelligence
11	Security Operations Features
12	Security Privacy and Ethics
13	Testing and Results
14	Discussion
15	Challenges and Resolutions
16	Limitations
17	Future Work
18	Conclusion
	Acknowledgments
	Conflict of Interest
	Data and Code Availability
	References
	Appendices

1 Introduction

1.1 Background

QR codes appear in payments, menus, tickets, login pages, advertisements, and public services. A person normally sees a web address before selecting it, but a QR code hides that address inside an image. The user often scans first and learns the destination only after the phone starts opening it. This creates a useful opening for phishing because the code can look legitimate even when the embedded address is not.

A malicious address may imitate a trusted organization, use a shortened link, move through several redirects, or rely on a newly created domain. Blacklists can stop known threats, but they may miss a site that has just appeared. A model can recognize patterns in the address, but it can also make mistakes. Certificate checks and URL rules provide useful additional evidence, yet neither one is enough on its own. QR Secures brings these signals together and explains the result before the user decides whether to continue.

1.2 Problem Statement

People need a practical way to reveal and assess the destination hidden inside a QR code before they open it. The assessment should combine more than one security signal, explain why an address looks risky, preserve a useful record of the scan, and support further review when a suspicious pattern affects more than one user.

1.3 Project Aim

The project aims to provide an accessible QR code and URL checking application that warns users about possible phishing, malware, scams, suspicious redirects, and social manipulation before they visit the destination.

1.4 Objectives

1. Compare deep learning approaches for malicious URL classification during the practicum stage.
2. Combine model output with URL rules and SSL certificate information in a hybrid decision process.
3. Provide camera scanning and manual URL checking through a clear mobile interface.
4. Explain each result with a score, a risk level, supporting indicators, and practical advice.
5. Store scan history and show useful trends without exposing one user to another user's records.
6. Allow users to report threats and incorrect results for community review.
7. Give analysts tools for investigations, notes, watchlists, and incident response.
8. Connect serious results to Slack so a security team can respond quickly.

1.5 Report Structure

The report first explains the project scope and technical background. It then describes the two development stages, application requirements, architecture, features, data design, and scan workflow. The later sections discuss community reporting, analyst tools, security controls, results, limitations, and future work.

2 Project Scope

2.1 Included Work

- QR code scanning through a device camera.
- Manual checking of a web address.
- URL structure, redirect, certificate, community, and language model analysis as described in the application material.

-
- Risk scores and classifications shown as Safe, Suspicious, or Malicious.
 - User accounts, personal scan history, dashboards, and account preferences.
 - Community reports for threats and incorrect classifications.
 - Analyst tools for filtering scans, adding notes, managing investigations, and maintaining watchlists.
 - Administrator tools for managing user roles.
 - Slack alerts and incident response settings.

2.2 Work Outside the Current Scope

The current project does not provide a formal guarantee that every malicious site will be detected. It does not replace a secure browser, endpoint protection, a web gateway, or professional investigation. The available material does not demonstrate full webpage content analysis, production scale testing, independent penetration testing, or deployment of the practicum CNN inside the current Base44 application. The source export needed to confirm several implementation details was unavailable.

2.3 Intended Users

User group	Main need	Application support
General user	Check an unfamiliar QR code or address	Scanner, Manual Scan, Results, History, Dashboard, Reports
Security analyst	Review patterns and related cases	Threat Hunting, notes, investigations, watchlists, reports
Administrator	Manage access and response settings	User Management, analyst tools, incident response settings

3 Background and Related Work

3.1 Known Address Lists

Traditional URL protection often compares an address with a list of known harmful sites. This approach is fast and useful for a threat that has already been reported. Its weakness is timing. A new domain or a changed path may not appear in the list when the first victims see it. Shortened links and redirects can also hide the final destination from a simple comparison.

3.2 Machine Learning for URLs

Machine learning can examine the text of an address and learn patterns that often occur in malicious URLs. Earlier systems relied on manually chosen features such as length, symbols, subdomains, and the use of an IP address. Deep learning can learn some of these patterns directly from the characters. During the practicum, the team compared CNN, GRU, and CNN LSTM models. The CNN gave the strongest reported performance and became the main classifier in the practicum detector.

3.3 Supporting Security Checks

A model score is more useful when it is considered with other evidence. QR Secures used URL rules to flag unusual length, suspicious symbols, risky words, domain concerns, and redirects. It also checked whether an SSL certificate was present and valid. A valid

certificate does not prove that a site is honest, but an invalid or expired certificate can strengthen other warning signs.

3.4 Language Model Analysis

The later Base44 application is described as using InvokeLLM with GPT 4o mini. A language model can organize several findings, identify possible phishing or social manipulation, and write an explanation that is easier for a general user to understand. It can also produce an incorrect explanation or assign too much weight to weak evidence. For that reason, the application should show the evidence and confidence rather than asking the user to trust a label alone.

3.5 Community Information

Community reports add human observations that may not yet appear in a public reputation service. A report can identify a phishing page, scam, malware link, suspicious redirect, or incorrect warning. This information becomes useful only after review. Poor moderation can create false accusations or allow an attacker to influence later results, so voting, analyst review, audit history, and expiry rules are important parts of the design.

4 Project Development

4.1 Practicum Stage

The practicum established the research foundation. The team prepared a labeled URL dataset, normalized the addresses, converted characters to numeric sequences, padded each sequence to a fixed length, and divided the data into training, validation, and testing groups. Three deep learning designs were compared. The chosen CNN was combined with URL rules and SSL checks through a weighted decision process.

The final practicum report described about 150,000 URLs and a split of 70 percent for training, 15 percent for validation, and 15 percent for testing. It reported 0.982 accuracy and 0.995 ROC AUC for the CNN. It also reported 48 correct classifications from 50 difficult cases for the hybrid detector. The original data split, saved model, and evaluation program are not included in the current repository, so these remain team reported results.

4.2 Capstone Stage

The capstone moved the work toward a usable application. Early capstone reports describe an Android package and a scan to verdict workflow based on the practicum detector. Later work expanded into the current Base44 application. The team refined the interface, reviewed several demonstration formats, improved result presentation, assigned presentation roles, and prepared the project for its defense.

4.3 Current Base44 Application

The Base44 version includes a React interface, authentication, data entities, backend functions, language model analysis, dashboards, community reporting, analyst tools, and Slack integration. This is a broader system than the practicum prototype. The source code could not be exported under the current Base44 plan, so the report distinguishes visible and documented behavior from code that has been independently reviewed.

5 System Requirements

5.1 Functional Requirements

Area	Requirement
------	-------------

Area	Requirement
Account access	The application must identify the user and load only the pages allowed for that role.
QR scanning	The user must be able to scan a QR code with the rear camera when the device supports it.
Manual checking	The user must be able to enter a web address without using a camera.
Analysis	The application must collect the available security signals and return a structured result.
Results	The result must show the address, score, risk level, confidence, evidence, and recommendation.
History	A user must be able to search and review previous scans.
Reporting	A user must be able to report a threat or an incorrect result.
Analyst work	An analyst must be able to filter scans, add notes, create investigations, and manage watchlists.
Alerts	An authorized user or verified automation must be able to send a security alert to Slack.
Administration	An administrator must be able to invite users and change roles.

5.2 Quality Requirements

Quality	Expected behavior
Clarity	A nontechnical user can understand the risk level and the reason for it.
Speed	The application keeps the user informed while checks are running and applies sensible time limits.
Privacy	Users do not see another user's private scans or account information.
Security	Backend functions verify the caller and validate every address before making an outbound request.
Reliability	A failed camera, network request, integration, or model response produces a clear error and a safe fallback.
Auditability	Important reports, role changes, investigation actions, and alerts can be reviewed later.
Accessibility	Text remains readable, controls have clear labels, and color is not the only risk indicator.

6 System Architecture

6.1 Technology

Layer	Documented technology
Interface	React 18, Tailwind CSS, Framer Motion, shadcn ui, and Radix UI
State and requests	TanStack Query version 5
Routing	React Router version 6
Platform	Base44 authentication, entities, integrations, and backend functions
Threat analysis	Base44 InvokeLLM described as using GPT 4o mini
QR scanning	html5 qrcode
Alerts	Slack through OAuth
Document export	jsPDF is listed in the technical report

6.2 Main Flow

Step	Action	Purpose
1	Capture QR content or accept a typed address	Obtain the value the user wants to check
2	Validate and identify the content	Separate a web address from text, contact data, or other QR content
3	Inspect redirects and the final domain	Reveal changes that may hide the true destination
4	Look for reviewed community reports	Add relevant observations from earlier scans
5	Run threat analysis	Assess the address and supporting context
6	Save the structured result	Create a history record for the user
7	Show the result and available actions	Help the user decide what to do next
8	Trigger authorized response rules	Notify a security team when a serious result meets a rule

6.3 Trust Boundaries

The device handles the camera, input, display, and user choices. The backend should handle contact with an unknown address, database access, language model requests, and Slack messages. The data layer must apply the user and role rules even if someone calls a backend

function directly. External services should receive only the minimum information required for their task.

7 Application Features

7.1 Splash

The Splash page introduces QR Secures and moves the user to Home after the opening sequence. The material also describes an option to skip the animation.

7.2 Home

Home is the main entry point after sign in. It shows the account, role, protection status, recent scan totals, camera and manual scanning options, and links to history, analytics, community reports, incident response, threat hunting, account settings, and user management when the role allows them.

7.3 Scanner

Scanner opens the device camera through html5 qrcode and looks for QR content in the camera frames. It prefers the rear camera and can offer a flashlight control when the device supports it. A manual entry link remains available if camera access fails or the user prefers typing.

7.4 Manual Scan

Manual Scan accepts a web address and sends it through the same documented security checks as a scanned address. Input validation should reject unsupported protocols, incomplete addresses, and values that could cause unsafe backend requests.

7.5 Results

Results loads a saved scan and presents the risk score, risk level, confidence, address, threat summary, phishing signs, social manipulation tactics, SSL information, URL structure, domain information, redirects, and recommendation when those fields are available. The user can copy the address, report the result, send an authorized Slack alert, scan again, or return home.

7.6 Dashboard

Dashboard summarizes the user's scan history. The documented charts include scan activity over time, risk distribution, common threat types, frequently scanned domains, and SSL status. Preferences are intended to control the visible widgets and date range, although the project history notes that some customization was incomplete.

7.7 History

History lists previous scans and supports text search, risk filters, scan type filters, date selection, sorting, mobile refresh, and opening a detailed result. Administrative deletion must be carefully authorized because it affects stored evidence.

7.8 Community Reports

Community Reports shows submitted threats and incorrect classifications. Users can filter the list by status and category. The records include the address, report type, category, explanation, evidence, votes, voters, date, and status.

7.9 Threat Hunting

Threat Hunting gives analysts and administrators a broader view of suspicious activity. It supports date, risk, threat type, score, and text filters. It also includes trends, a scan table, detailed review, annotations, investigations, and watchlists.

7.10 Automation Settings

Automation Settings stores conditions for incident response. A rule can react to a malicious, suspicious, or high score result and request an action such as a Slack or email notification. Older materials call this feature Playbooks, but the Playbooks page was later removed.

7.11 Account

Account shows the user name, email address, role, join date, and available preferences. Any editable information should be validated and limited to the signed in account.

7.12 User Management

User Management is intended for administrators. It supports user search, invitations, and role changes. Role changes must be checked on the backend because a hidden page alone does not stop an unauthorized request.

8 Data Design

8.1 Scan History

Each completed scan creates a history record. The technical material lists the scan type, extracted content, content type, risk score, risk level, confidence, SSL result, detected threats, phishing indicators, social manipulation tactics, summary, detailed analysis, favorite status, and creation information.

8.2 Threat Reports

A threat report records the address, whether the user is reporting a threat or an incorrect result, the category, description, evidence, status, vote totals, and voters. The files disagree on whether five votes change the status automatically or whether an analyst must also approve it. The implementation should use a clear rule and preserve who made each decision.

8.3 Investigations and Notes

An investigation has a title, status, priority, assigned members, linked scans, findings, and resolution notes. Scan annotations let an analyst attach a note, finding, incorrect result, or investigation reference to a scan. Together, these records preserve the reasoning behind an analyst decision.

8.4 Watchlists and Rules

A watchlist entry identifies a domain, address pattern, threat type, or keyword. It can record whether alerts are enabled, where a notification should go, and how often the item matched. An automation rule defines a trigger, a minimum score, and the actions to request. Alert addresses and connector settings require careful access control.

8.5 User Preferences

User preferences can store dashboard order, visible widgets, pinned measures, and a default date range. These settings improve convenience but should not change security permissions or the underlying scan result.

9 Scan and Analysis Workflow

9.1 Camera Capture

The scanner requests camera permission after the user chooses to scan. It asks for the rear camera, starts the preview, and checks frames for a QR pattern. When a code is decoded, the scanner stops the camera and passes the text to the next step. The page should release camera resources when the user leaves or when an error occurs.

9.2 Content Validation

The application decides whether the decoded value is a web address or another type of content. A web address enters the full security process. Plain text, contact data, WiFi details, phone numbers, and email addresses need separate handling so the application does not treat every QR code as a website.

9.3 Redirect Resolution

The project presentation describes a backend function that reads redirect responses one hop at a time, records each address and status, stops after ten hops, and compares the first domain with the final domain. This avoids opening the chain in the user's browser during analysis. It does not remove all risk because the backend still contacts an outside system. The function must reject private addresses, check DNS results at every hop, restrict protocols, set time limits, and avoid downloading large responses.

9.4 Community Check

The application looks for reviewed reports that match the destination. A confirmed threat can add useful context. A confirmed incorrect result can warn the analysis that earlier users found the address legitimate. Both kinds of reports need a clear age, source, and review status so old or manipulated reports do not control a new decision.

9.5 Threat Analysis

The Base44 application is described as sending the original address, final destination, redirect details, and community context to InvokeLLM. It expects a structured response with a score, level, confidence, summary, indicators, tactics, certificate result, and recommendation. The application must validate the returned structure and use a safe fallback when the response is incomplete.

9.6 Saving and Displaying the Result

After analysis, the application saves the result in Scan History and opens Results using the new record identifier. The Results page reads that record, parses any stored redirect details, and displays the available evidence. If saving fails, the user should see a clear message and the application should avoid presenting an unsaved result as if it were complete.

10 Community Intelligence

10.1 Reporting

A user can open the report form from Results and choose a threat or incorrect result. The form asks for a category, explanation, and optional evidence. The application stores the report as Pending until the review rule is satisfied.

10.2 Voting and Review

The interface uses immediate visual updates when an analyst votes so the page feels responsive. The server still decides whether the vote is valid. It must prevent duplicate

votes, confirm the reviewer role, and return the correct totals if the request fails. A review log should show who changed the status and why.

10.3 Feedback into Later Scans

Reviewed community evidence can be added to later analysis of the same address. The project history records a fix that was meant to send both confirmed threats and confirmed incorrect results to the analysis prompt. This closes an important gap, but the deployed behavior should be tested with controlled addresses before it is presented as complete.

10.4 Misuse Prevention

A community system can be abused to harm a legitimate site or protect a malicious one. Useful controls include verified accounts, limited voting rates, analyst review, audit history, report expiry, conflict handling, and detection of coordinated voting. A report description must also be treated as untrusted text when it is included in a model request.

11 Security Operations Features

11.1 Threat Hunting

Threat Hunting lets an analyst search beyond one result. Filters help find a time period, high scores, a threat type, or a repeated domain. A trend chart can show whether suspicious activity is increasing, while the scan table gives access to the evidence behind each point.

11.2 Investigations

An investigation groups related scans into a case. The title and priority help organize work, assigned members show responsibility, linked scans preserve the evidence, and findings record what the team learned. Status values such as Open, In Progress, Resolved, and Closed show the current stage.

11.3 Watchlists

Watchlists help analysts monitor a domain, pattern, keyword, or threat type. A match count can show repeated activity. The material includes an alert setting and email field, but the project review notes that match notifications may not be fully connected. This should be confirmed before relying on it.

11.4 Slack Alerts

The application material lists functions for sending a manual alert, sending an automatic threat alert, listing Slack channels, synchronizing results, and posting malware alerts. An alert can include the risk level, address, score, and summary. The payload should avoid personal data and private links unless the receiving channel is approved for that information.

11.5 Automated Rules

An authorized analyst can define a condition and an action. The backend can evaluate the rule after a scan is saved so the user does not wait for Slack or email. The system must distinguish a real Base44 automation event from a direct request sent by an unknown caller.

12 Security Privacy and Ethics

12.1 Authentication and Authorization

Every protected page should require sign in, but sign in alone does not grant access to every record. The backend must check ownership for personal scans and check the analyst or administrator role for broader operations. The application added row level rules for several entities during the September 2026 security review, and those rules need regression testing.

12.2 Open Backend Function Issue

The same security review identified four functions that did not verify who was calling. They were the automatic Slack alert, threat detection handler, malware alert, and Slack synchronization functions. Anyone who knew an exposed function address could potentially trigger an outside request or message. The current Base44 plan did not permit the functions to be changed. This remains a release blocker. Each function must accept either an authenticated and authorized user or a signed internal automation request before it reads data, uses a secret, or contacts Slack.

12.3 Redirect Safety

A redirect checker can become a path into internal networks if it accepts any destination. The function must block local names, loopback addresses, private ranges, link local addresses, cloud metadata addresses, and unsupported protocols. It must repeat the check after every DNS lookup and redirect because an attacker can change the target during the chain.

12.4 Privacy

A scanned address can contain personal or confidential information. Password reset links, invitation links, document shares, internal hostnames, and tracking parameters may identify a person or grant access. The interface should warn users not to submit such links. Stored content needs a retention period, deletion process, access log, and limited visibility. Slack alerts should remove unnecessary query values and personal details.

12.5 Explainable Results

A user should see why the application assigned a risk level. The explanation should separate observed facts, such as a domain change, from model interpretation, such as suspected impersonation. Confidence must not be presented as a measured probability unless the system has been calibrated. Low confidence should lead to a clear request for caution and manual review.

12.6 Responsible Testing

The project is defensive. Testing should use controlled domains, harmless QR codes, and approved systems. The application should not probe ports, submit forms, execute page scripts, or collect page content without a clear need and authorization. Reports about a real domain should be reviewed before public display.

13 Testing and Results

13.1 Practicum Model Results

Model	Accuracy	Precision	Recall	F1 score	ROC AUC
CNN	0.982	0.975	0.989	0.982	0.995
GRU	0.975	0.968	0.982	0.975	0.991
CNN LSTM	0.979	0.971	0.987	0.979	0.993

These values come from the final practicum report. The CNN was selected because it had the strongest reported result across the listed measures. The training data, fixed test identifiers, saved model, and evaluation program are not present in the repository, so the table records the team result rather than an independently reproduced benchmark.

13.2 Practicum Hybrid Result

The practicum report states that the hybrid detector classified 48 of 50 difficult URLs correctly, which equals 96 percent. It also describes cases where certificate or URL evidence changed a borderline result. The 50 addresses and expected answers are not included, so the result cannot yet be repeated.

13.3 Capstone Application Review

Area	Evidence reviewed	Current conclusion
Core navigation	Live Base44 preview and demonstration video	Home, Scan, History, Account, Dashboard, Reports, Incident Response, Threat Hunting, and User Management were visible according to role.
Camera workflow	Demonstration video and project history	The application presents a camera scanner and manual alternative.
Results and history	Demonstration video and technical report	Risk results, scan records, filters, and charts are part of the user flow.
Community reports	Technical report and Base44 history	Threat and incorrect result reporting exist, but final moderation behavior needs source verification.
Security changes	Base44 security activity	Frame protection and row level rules were added. Four backend functions remain unresolved.

13.4 Claims That Need More Evidence

Later presentation material states that the language model application reached 94 percent phishing accuracy, detected every redirect attack in a set of 50, improved on older databases by 26 percentage points, reduced the longest scan time from 8 seconds to 3.2 seconds, and completed more than 28 automated responses. The supplied files do not include the test set, comparison method, logs, or evaluation program for these claims. They are therefore excluded from the report conclusions.

14 Discussion

14.1 Main Contribution

The strongest contribution is the complete workflow rather than one model score. QR Secures connects QR capture, address analysis, an understandable result, personal history, community review, analyst investigation, and team alerts. This gives the project practical value and shows how a research detector can develop into a broader security application.

14.2 Value of Multiple Signals

Each signal answers a different question. The URL text can reveal suspicious patterns. Certificate information can show a connection problem. Redirects can reveal a change in destination. Community reports can add recent human observations. A language model can summarize the evidence. The final decision is more useful when the application shows how these signals agree or disagree.

14.3 Human Review

Security results affect trust, so people need a way to challenge them. Reporting an incorrect result, adding analyst notes, and recording an investigation provide that path. The application will be stronger when a confirmed correction has a clear and measured effect on later scans without letting one user override the system.

14.4 Practical Deployment

The Base44 application lowers the effort needed to demonstrate and use the system. At the same time, production deployment requires stronger evidence and controls than a course demonstration. The open backend function issue, redirect protections, data retention, alert privacy, model evaluation, and operational monitoring should be completed before the application is treated as a production security service.

15 Challenges and Resolutions

Challenge	Response	Remaining work
Keeping practicum and capstone claims consistent	The report separates the CNN detector from the later Base44 application.	Add source and deployment evidence if the two systems are integrated.
Presenting complex results clearly	The interface uses a score, level, evidence, and recommendation.	Test comprehension with users and improve low confidence wording.
Camera and mobile behavior	The project added a mobile scanner, manual entry, responsive selections, and saved tab position.	Test more browsers, devices, permissions, and camera failures.
Fast interface updates	Voting and settings use immediate local updates with rollback on failure.	Confirm server results always replace temporary values.
False positive handling	Users can report an incorrect result and the project history records a feedback fix.	Verify the deployed behavior and add analyst correction history.
Backend security findings	Row level rules and frame protection were added.	Secure the four open functions and repeat the security review.
Source access	The report relies on reviewed documents, the live preview, and project history.	Export and review the source when the Base44 plan permits it.

16 Limitations

9. The application can miss a threat or incorrectly warn about a legitimate address.
10. The practicum results cannot be reproduced without the original data split, model file, and evaluation code.
11. The later application performance claims do not include enough evidence for independent verification.
12. The Base44 source could not be exported under the current plan.
13. Four backend functions remain without confirmed caller verification.
14. A HEAD request may not follow the same path as a normal browser request.

-
15. A valid certificate does not prove that a website is trustworthy.
 16. Community reports can become stale or can be manipulated without strong review controls.
 17. Language model explanations can be inaccurate, inconsistent, or overly confident.
 18. The application does not provide confirmed visual analysis of the destination webpage.
 19. Storing full URLs can expose private tokens or personal information.
 20. The project material does not show independent penetration testing or large scale production testing.

17 Future Work

17.1 Immediate Security Work

The first priority is to secure the four backend functions. Each function needs authentication for user requests and a signed method for internal automation. Redirect handling needs protection against private network access. The team should then repeat the security review and test every role with direct backend requests.

17.2 Evidence and Evaluation

The project should publish a reproducible evaluation package with dataset sources, fixed splits, model settings, saved hashes, test programs, safe challenge cases, and benchmark logs. The Base44 application needs a separate evaluation that measures detection quality, false warnings, response time, consistency, and user understanding.

17.3 Product Improvements

Useful improvements include an analyst correction workflow, a carefully controlled trusted domain list, clear low confidence warnings, scan deduplication, batch checking for analysts, investigation export, working watchlist notifications, report expiry, and complete dashboard preferences. Each feature should preserve a history of who changed a decision and why.

17.4 Research Directions

Future research could compare the practicum CNN with the language model process on the same fixed data. It could also study webpage content, visual impersonation, privacy preserving threat sharing, local analysis for restricted environments, and calibrated confidence. Any new comparison should use a transparent method and a test set that does not leak into model development.

18 Conclusion

QR Secures addresses a simple but important problem: a QR code can hide a dangerous destination until the user opens it. The practicum showed the reported value of a character level CNN supported by URL rules and SSL checks. The capstone developed that work into a broader application with scanning, explanations, history, community reports, investigation tools, watchlists, role management, and Slack alerts.

The project demonstrates a useful direction for preventive QR security, but the evidence must remain tied to the correct stage. The CNN metrics belong to the practicum detector. The current Base44 application needs its own reproducible evaluation. It also needs the remaining backend security issue fixed before production use. With those steps complete, QR Secures could provide a stronger and more accountable way for users and security teams to examine unfamiliar QR destinations.

Acknowledgments

The team thanks Dr. Adnan for his supervision, feedback, and guidance throughout the practicum and capstone work. The team also thanks the College of Computing and Information Technology for the academic setting and resources that supported the project.

Conflict of Interest

The authors declare no conflict of interest.

Data and Code Availability

The practicum used publicly available malicious and benign URL data described in the earlier report. The fixed data split, trained model, and evaluation code are not included in this repository. The current application is hosted on Base44 at <https://qrsecures.base44.app>. Source export was unavailable under the current Base44 plan. Academic files may be shared with the supervisor or review panel as required by the course.

References

- [1] I. Goodfellow, Y. Bengio, and A. Courville. Deep Learning. MIT Press, 2016.
- [2] M. Le, Q. Pham, O. Sahoo, and S. Hoi. URLNet Learning a URL Representation with Deep Learning for Malicious URL Detection. 2018.
- [3] S. J. Bu and S. B. Cho. Deep Character Level Anomaly Detection Based on a Convolutional Autoencoder for Malicious URL Detection. Electronics, volume 10, issue 12, 2021.
- [4] F. Türk. Malicious URL Detection with Advanced Machine Learning and Optimization Based Hybrid Methods. Applied Sciences, volume 15, issue 18, 2025.
- [5] B. C. Ujah Ogbuagu and coauthors. A hybrid deep learning technique for spoofing website URL detection in real time applications. Journal of Electrical Systems and Information Technology, 2024.
- [6] TensorFlow Developers. TensorFlow machine learning platform. <https://www.tensorflow.org>
- [7] F. Pedregosa and coauthors. Scikit learn Machine Learning in Python. Journal of Machine Learning Research, volume 12, 2011.
- [8] Base44. Application platform documentation. <https://docs.base44.com>
- [9] html5 qrcode project documentation. <https://github.com/mebjas/html5-qrcode>
- [10] Slack. OAuth and messaging documentation. <https://api.slack.com>

Appendix A Page and Feature Summary

Page	Main purpose	Typical access
Splash	Introduce the application and open Home	Signed out or signed in entry
Home	Show scan options, totals, shortcuts, and role tools	All signed in users
Scanner	Capture QR content through the camera	All signed in users
Manual Scan	Check a typed web address	All signed in users
Results	Explain a saved assessment and offer follow up actions	Owner of the scan and authorized review roles
Dashboard	Summarize personal scan activity	All signed in users
History	Search and review previous scans	All signed in users
Community Reports	Submit and review threat or incorrect result reports	Submission for users, review for analysts and administrators
Threat Hunting	Search wider activity and manage investigations	Analyst and administrator
Automation Settings	Configure response conditions and alerts	Analyst and administrator
Account	View profile and preferences	All signed in users
User Management	Invite users and change roles	Administrator

Appendix B Data Entity Summary

Entity	Purpose	Important information
Scan History	Store a completed scan	Content, type, score, level, confidence, SSL result, indicators, summary, details
Threat Report	Record community evidence	Address, report type, category, explanation, evidence, votes, voters, status
Investigation	Group related scans	Title, status, priority, assigned members, scan references, findings, resolution
Watchlist	Monitor an indicator	Indicator type, value, alert choice, contact, match count
Automation Rule	Request an incident response	Trigger, minimum score, requested actions

Entity	Purpose	Important information
Scan Annotation	Attach analyst reasoning	Scan reference, note type, content, status, tags
User Preferences	Store display choices	Widgets, pinned measures, date range

Appendix C Release Checklist

- Secure the four backend functions and verify internal automation signatures.
- Test row level access for a user, analyst, administrator, and direct API request.
- Block private and reserved addresses during every redirect step.
- Remove secrets and private data from logs and Slack messages.
- Test camera denial, camera failure, network timeout, malformed model output, and database failure.
- Verify threat reports and incorrect result reports affect later scans as intended.
- Measure false warnings, missed threats, response time, and user understanding with a fixed test plan.
- Define data retention, deletion, backup, and incident response procedures.
- Export and review the Base44 source when the plan allows it.
- Repeat the security scan and document the final result.